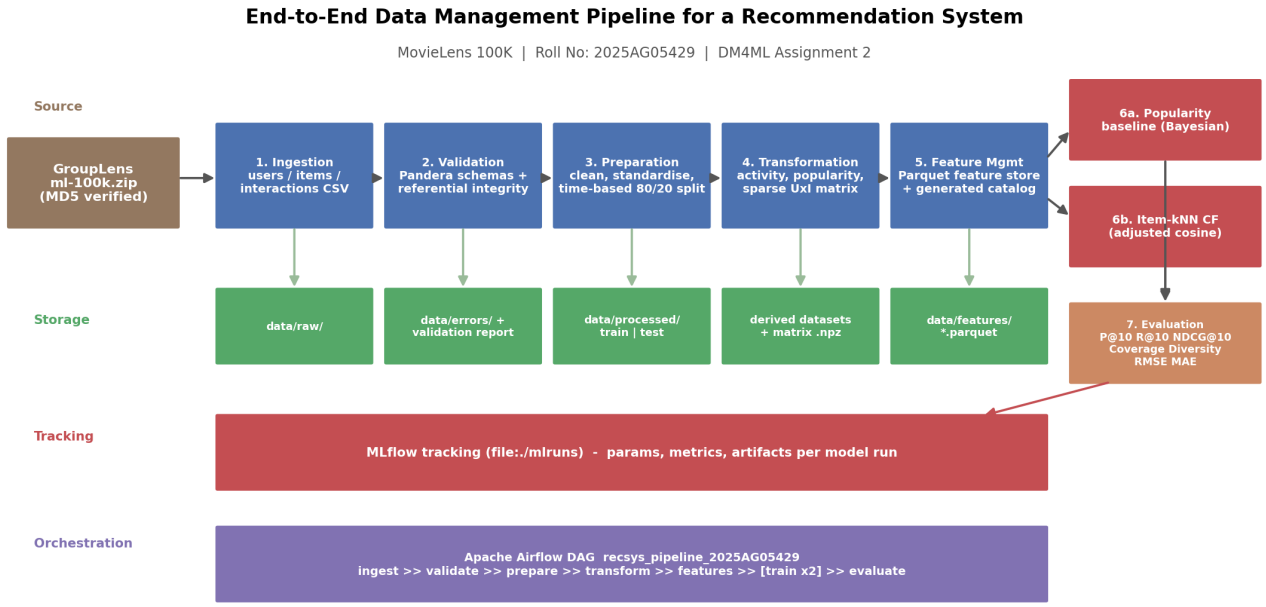
End-to-End Data Management Pipeline for a Recommendation System

Student	SUTARIYA PARTH POPATBHAI
BITS ID / Roll No	2025AG05429
Course	Data Management for Machine Learning (AIMLZG529)
Assignment	Assignment 2 - Recommendation pipeline (10% weightage)
Dataset	MovieLens 100K (GroupLens; 943 users, 1,682 movies, 100,000 explicit ratings)
Repository	GitHub (link submitted on the LMS) - fully reproducible, see README.md

1. Objective and design overview

The project implements the complete chain **ingestion** → **validation** → **preparation** → **transformation** → **feature management** → **training** → **evaluation** → **orchestration** for a movie recommender. Design priorities were reproducibility (pinned dependencies, checksum-verified dataset, one-command run), modularity (each stage is an independent module with its own entry point, wired together identically by the sequential runner and by the Airflow DAG) and honest evaluation (a time-based split, features computed strictly on the training window). All hyper-parameters live in `configs/pipeline.yaml`.

2. Pipeline architecture



3. Stage-by-stage design decisions

Stage	Decisions and evidence
Ingestion	ml-100k.zip downloaded programmatically from GroupLens and verified against the pinned official MD5 before extraction; the three entities are rewritten as canonical CSVs (users / items / interactions). Summary: reports/ingestion_summary.json.
Validation	Pandera schemas + custom checks: missing values, dtype coercion, duplicate keys and duplicate events, rating in [1,5], timestamp in [1995, now], gender/genre domains, and referential integrity of interactions against both dimension tables. 32 checks executed; invalid rows are quarantined to data/errors/ with an error_reasons column (unit-tested on synthetic dirty data). All 100,000 interactions passed.
Preparation	Formats standardised (dates to ISO, categorical normalisation, release year recovered from titles), re-ratings resolved keeping the latest. Chronological 80/20 split at 1998-03-07: train=80,003, test=19,997; 192 of 301 test users are cold-start (never seen in train).

Stage	Decisions and evidence
Transformation	Derived datasets built from the train window only: user activity summary, item popularity with Bayesian damped mean (m=20), and a sparse 751x1682 user-item matrix (density 6.33%).
Features	24 user/item/interaction features computed 'as of' the last train timestamp and served from a Parquet feature store (src/features/store.py) that is reused by training AND prediction - no train/serve skew. The feature catalog (reports/feature_catalog.md) is generated from the code registry, so docs cannot drift.
Training	Two models tracked in MLflow (params, metrics, artifacts, one run per model): (a) popularity baseline ranking by the feature store's Bayesian item rating; (b) personalised item-kNN collaborative filtering - adjusted-cosine similarity, significance shrinkage (n/(n+100)), top-30 neighbours, and Netflix-style baseline biases for rating prediction.
Evaluation	Protocol: for every test user with train history, recommend K=10 unseen items; relevant = test rating >= 4. Metrics: Precision@10, Recall@10, NDCG@10, catalog coverage, intra-list (genre) diversity, plus RMSE/MAE because explicit ratings are predicted. Evaluation metrics are appended to each model's MLflow training run.
Orchestration	Apache Airflow 2.10.5 (standalone mode, as in the lab session) runs DAG recsys_pipeline_2025AG05429: ingest >> validate >> prepare >> transform >> features >> [train_popularity || train_item_knn] >> evaluate, with retries and a weekly schedule. Airflow lives in its own virtualenv; each task executes the stage module in the project venv via BashOperator. src/pipeline.py offers the same flow without Airflow.

4. Model evaluation results

Model	P@10	R@10	NDCG@10	Coverage	Diversity	RMSE	MAE
popularity_baseline	0.0888	0.0914	0.1100	0.0297	0.7402	1.0397	0.8367
item_knn_cf	0.1112	0.0603	0.1279	0.0842	0.6957	1.0312	0.8001

The personalised item-kNN model improves over the baseline on Precision@10 (+25%), NDCG@10 (+16%), catalog coverage (2.8x) and on both rating-error metrics, confirming that collaborative signals add value beyond popularity. The baseline retains the higher Recall@10: under a chronological split the most recent window is dominated by widely watched blockbusters, which a popularity list retrieves by construction - an expected and well-documented property of time-aware evaluation, not an artefact of the pipeline. Diversity is comparable (0.70 vs 0.74) because the kNN lists concentrate inside the user's preferred genres, which is the desired personalisation behaviour. Sample per-user recommendations are committed in reports/evaluation/sample_recommendations.csv.

5. Reproducibility

Fresh-Ubuntu run: `bash scripts/setup.sh && bash scripts/run_pipeline.sh` rebuilds every artifact (validation report, feature store, models, MLflow runs, evaluation tables) deterministically in about two minutes; `bash scripts/setup_airflow.sh` starts the orchestrated version. Pinned requirements.txt, pinned dataset checksum, pytest unit tests for the validation quarantine, metrics and models, and stage idempotency guarantee identical results across machines.

6. Limitations and future work

(i) Cold-start users in the test window fall back to popularity; demographic features from the store could drive a content-based fallback instead. (ii) MovieLens 100K is small - the dense item-item similarity would need approximate nearest neighbours at catalogue scale. (iii) A single chronological split is used; rolling-origin evaluation would give confidence intervals. (iv) MLflow runs locally with a file store; a remote tracking server and model registry would be the next step towards deployment, together with scheduled retraining already supported by the weekly Airflow DAG.

Evidence committed in the repository: validation report (md/json), quarantine folder, feature catalog, model comparison table and chart, MLflow run screenshots, Airflow DAG file and successful-run screenshot, pipeline log.

Roll No: 2025AG05429 | DM4ML Assignment